

力學數值工具包 / Mechanics Numerical Toolkit

這個工具包為古典力學課程提供一個小型、共用的計算介面。它不是程式設計作業，也不是數值分析專題。重點是幫助你執行物理模型、把模型轉成 acceleration function，並使用 numerical integrator 產生與解讀軌跡。

This toolkit provides a small, shared computational interface for the Classical Mechanics course. It is not a programming assignment and it is not a numerical-analysis project. The goal is to help you run a physical model, turn it into an acceleration function, and use a numerical integrator to generate and interpret trajectories.

1. 這個工具包的用途 / What this toolkit is for

工具包提供四種 numerical integrator：

The toolkit provides four numerical integrators:

- semi-implicit Euler
- Verlet
- velocity Verlet
- RK4 (fourth-order Runge-Kutta)

你不需要自行實作這些方法。你需要理解如何把你的 physical model 連接到正確的 acceleration interface，並解讀輸出的 $\mathbf{x}(\mathbf{t})$ 與 $\mathbf{v}(\mathbf{t})$ 。

You do not need to implement these methods yourself. You do need to understand how to connect your physical model to the correct acceleration interface and how to interpret the resulting $\mathbf{x}(\mathbf{t})$ and $\mathbf{v}(\mathbf{t})$.

2. 快速開始 / Quick Start

如果你剛下載這個資料夾，最短路徑是：

If you just downloaded this folder, the shortest path is:

1. 建立本地 virtual environment。
2. 安裝相依套件。
3. 執行 getting_started.py。
1. Create a local virtual environment.
2. Install the dependencies.
3. Run getting_started.py.

以下有逐步說明。

Step-by-step instructions are below.

3. 安裝與環境設定 / Installation and environment setup

在此資料夾建立本地 virtual environment：

Create a local virtual environment in this folder:

```
python -m venv .venv
```

啟用環境：

Activate it:

- Windows PowerShell

```
.\.venv\Scripts\Activate.ps1
```

- Windows Command Prompt

```
.venv\Scripts\activate.bat
```

- macOS / Linux

```
source .venv/bin/activate
```

安裝需要的套件：

Install the required packages:

```
python -m pip install -r requirements.txt
```

這個工具包用 `numpy` 進行計算，並用 `matplotlib` 繪圖。

This toolkit uses `numpy` for computation and `matplotlib` for plotting.

4. 執行 getting_started.py / Run getting_started.py

安裝完成後，執行：

After installation, run:

```
python getting_started.py
```

這個腳本是一個簡短的入門示例，使用一維等加速度自由落體：

This script is a short onboarding example using one-dimensional free fall with constant acceleration:

```
\[  
x''=-g  
\]
```

它展示基本 workflow、畫出數值結果，並與解析解比較。

It shows the basic workflow, plots the numerical result, and compares it with the analytic solution.

5. 基本思考流程 / Basic mental model

典型計算流程如下：

A typical calculation looks like this:

physical model

-> acceleration function

-> initial conditions

-> time grid

-> numerical integrator

-> $x(t)$, $v(t)$

-> plots and physical checks

實際上，你先用你自己的語言描述物理，再把它轉成像 **acceleration** 這樣的函式，選擇初始位置與初始速度，設定 time grid，呼叫 integrator，最後檢查回傳陣列。

In practice, you describe the physics in your own words, turn it into a function named something like **acceleration**, choose initial position and velocity, choose a time grid, call an integrator, and then inspect the returned arrays.

一般情況下，你不需要修改 `mechanics_integrators.py`。

You normally do not need to modify `mechanics_integrators.py`.

6. 如何定義 acceleration / How to define acceleration

integrator 會在每個 time step 呼叫你的 acceleration function。這個函式回傳你的 physical model 所對應的瞬時加速度。

The integrators call your acceleration function at each time step. The function returns the instantaneous acceleration implied by your physical model.

對純量問題，函式可以回傳純量。

For scalar problems, the function can return a scalar.

對向量問題，回傳的 acceleration 必須與位置向量有相同 shape。

For vector problems, the returned acceleration must have the same shape as the position vector.

範例：

Examples:

```
def acceleration(t, x, v):  
    return -9.8  
  
def position_acceleration(t, x):  
    return -k * x / m
```

7. 初始條件與 time grid / Initial conditions and time grid

你需要一個初始位置與一個初始速度：

You need an initial position and an initial velocity:

- x_0
- v_0

它們可以是純量或 NumPy 陣列，但 shape 必須一致。

These may be scalars or NumPy arrays, but they must have the same shape.

你也需要一維 time grid, 例如：

You also need a one-dimensional time grid such as:

```
t = np.linspace(0, 2.0, 201)
```

time grid 應該等間距。工具包會把相鄰時間點間距當作 timestep。

The time grid should be evenly spaced. The toolkit uses the spacing between adjacent values as the time step.

8. 呼叫 integrator 與解讀 `x`、`v` / Calling an integrator and interpreting `x` and `v`

所有 integrator 都會回傳兩個陣列：

All integrators return two arrays:

- **x**: 每個時間點的位置
- **v**: 每個時間點的速度
- **x**: position at each time point
- **v**: velocity at each time point

範例：

Example:

```
x, v = rk4(acceleration, t, x0, v0)
```

若 **t** 長度為 **N**, 則 **x** 與 **v** 在時間軸上的長度也為 **N**。純量問題時它們是一維陣列；向量問題時, 每個時間切片 shape 與 **x0**、**v0** 相同。

If **t** has length **N**, then **x** and **v** have length **N** along the time axis. For a scalar problem, they are one-dimensional arrays. For a vector problem, each time slice has the same shape as **x0** and **v0**.

9. 可用的 integrator / Available integrators

semi-implicit Euler

```
x, v = semi_implicit_euler(acceleration, t, x0, v0)
```

使用 **acceleration(t, x, v)**。

Uses **acceleration(t, x, v)**.

Verlet

```
x, v = verlet(position_acceleration, t, x0, v0)
```

使用 **position_acceleration(t, x)**。

Uses **position_acceleration(t, x)**.

velocity Verlet

```
x, v = velocity_verlet(position_acceleration, t, x0, v0)
```

使用 `position_acceleration(t, x)`。

Uses `position_acceleration(t, x)`.

RK4

```
x, v = rk4(acceleration, t, x0, v0)
```

使用 `acceleration(t, x, v)`。

Uses `acceleration(t, x, v)`.

10. 適用性與 acceleration-function 需求 / Applicability and acceleration-function requirements

這個工具包使用兩種 acceleration interface：

This toolkit uses two different acceleration interfaces:

- `a(t, x, v)`：用於一般一階 state form
- `a(t, x)`：用於 position-based 的 Verlet family
- `a(t, x, v)` for the general first-order state form
- `a(t, x)` for the position-based Verlet family

這表示本工具包中的標準 Verlet 與 velocity Verlet 實作，預期 acceleration 只依賴時間與位置。它們不是可直接處理顯式 velocity-dependent force 的通用介面。

That means the standard Verlet and velocity Verlet implementations in this toolkit expect acceleration to depend on time and position only. They are not general interfaces for forces with explicit velocity dependence.

工具包也會檢查回傳 acceleration 的 shape 是否與 state 相容。向量問題中，acceleration 必須與位置 shape 完全一致。

The toolkit also checks that the returned acceleration has a shape compatible with the state. For vector problems, the acceleration must match the position shape exactly.

11. timestep 的考量 / Time step considerations

timestep 是 computational experiment 的一部分。較小 timestep 常會提高數值結果精度，但也代表更多計算量。

The time step is part of the computational experiment. A smaller step often gives a more accurate numerical result, but it also means more computation.

對學生專題，主要問題是：

For a student project, the main questions are:

- timestep 是否足夠小，讓結果穩定且在物理上合理？
- 改變 timestep 會不會明顯改變答案？
- 多次執行之間是否比較同一個物理量？
- Is the step small enough that the result is stable and physically reasonable?
- Does changing the step noticeably change the answer?

- Are you comparing the same physical quantity across runs?

你不需要把這件事變成數值分析研究。

You do not need to turn this into a numerical-analysis study.

12. 在工具包中使用 generative AI / Using generative AI with the toolkit

你可以自由使用 generative AI，但應把本工具包檔案視為運作方式的主要來源。

You may use generative AI freely, but you should treat this toolkit as the primary source for how it works.

用於學習工具包的範例 prompt：

Example prompt for learning the toolkit:

```
Read README.md and getting_started.py from this mechanics numerical toolkit. Explain the basic usage of the toolkit at a beginner level. Explain what an acceleration function is, what t, x0, and v0 mean, what an integrator returns, and how getting_started.py works. Also explain the difference between the acceleration interfaces used by semi-implicit Euler and RK4 versus the interfaces used by Verlet and velocity Verlet. Use the supplied files as the primary source. Do not rewrite mechanics_integrators.py. Do not invent undocumented APIs. If something is unclear, say so rather than guessing.
```

用於連接自己 physical model 的可選 prompt：

Optional prompt for a student connecting their own physical model:

```
I am trying to connect my own classical-mechanics model to this toolkit. First help me check the physical equation, what the acceleration function should depend on, whether the chosen integrator interface is applicable, and what initial conditions and time interval I need to choose. Do not immediately write the full experiment. Do not replace my physical reasoning. Do not rewrite the integrator implementation.
```

13. 常見問題與排查 / Common problems and troubleshooting

- 若 Python 找不到 `numpy` 或 `matplotlib`，請重新確認 virtual environment，並再執行一次 `python -m pip install -r requirements.txt`。
- 若沒有出現繪圖視窗，範例仍可能已正確執行；某些環境使用非互動式繪圖 backend。
- 若你傳入向量 state，請確認 acceleration 回傳同 shape 的向量。
- 若看到 acceleration shape 相關錯誤，請檢查你的函式是否在需要向量時回傳了純量，或反之。
- If Python cannot find `numpy` or `matplotlib`, re-check the virtual environment and run `python -m pip install -r requirements.txt` again.
- If a plot window does not appear, the example may still have run correctly; some environments use a non-interactive plotting backend.
- If you pass a vector state, make sure your acceleration returns a vector of the same shape.
- If you see an error about acceleration shape, check whether your function is returning a scalar where a vector is needed, or vice versa.

14. 現有拋體與單擺範例 / Existing projectile and pendulum examples

example_projectile.py 與 example_pendulum.py 是參考範例。

The files example_projectile.py and example_pendulum.py are reference examples.

- `getting_started.py` 是簡短的入門示例。
- `example_projectile.py` 展示含線性阻力的拋體問題。
- `example_pendulum.py` 展示無阻尼的非線性單擺。
- `getting_started.py` is a short tutorial example.
- `example_projectile.py` shows a projectile problem with linear drag.
- `example_pendulum.py` shows a nonlinear pendulum without damping.

這些範例不是 A2 作業本身。對於 A2，你會圍繞自己選擇的 physical model 設計你的 computational experiment。

These examples are not the A2 assignment itself. For A2, you will design your own computational experiment around the physical model you choose.